

PROYECTO FINAL DE GRADO – COSTURAS CON MIMO WEB VUE/DJANGO/POSTGRES – MANUAL DE CÓDIGO



Alumno: Pablo Díaz Diez

Profesor: Jose María Herrero

Fecha: 07/04/2026

2º Desarrollo de Aplicaciones Multiplataforma

1. Índice

1.	Índice	2
2.	Índice de ilustraciones.....	2
3.	Tecnologías empleadas	4
3.1.	Frontend: Vue.js y Vite	4
3.2.	Backend: Django y Python.....	4
3.3.	Base de Datos: PostgreSQL.....	4
3.4.	Pagos y emails	4
4.	Herramientas empleadas	5
4.1.	Visual Studio Code	5
4.2.	pgAdmin 4	5
5.	Organización de carpetas	6
5.1.	Frontend.....	6
5.2.	Backend	8
6.	Diseño de Datos.....	10
7.	Explicación de código.....	11
7.1.	Login y seguridad.....	11
7.2.	Encargos.....	13
7.3.	Pago con Stripe	14
7.4.	Correos con mailtrap	15
8.	Problemas	15
9.	Puesta en marcha	16

2. Índice de ilustraciones

Ilustración 1 Visual Studio Code	5
Ilustración 2 pgAdmin 4	6
Ilustración 3 Organización Frontend	8
Ilustración 4 Organización Backend.....	10
Ilustración 5 Models.py.....	11

Ilustración 6 Método index.js	12
Ilustración 7 Rutas	13
Ilustración 8 Encargo View.py	14
Ilustración 9 Pago carrito.vue	14
Ilustración 10 Envío de correos.....	15

3. Tecnologías empleadas

Para el desarrollo de "Costuras con Mimo", planteé desde el inicio una arquitectura cliente-servidor desacoplada. Esto me permite gestionar la interfaz de usuario de forma independiente a la lógica del servidor. Estas son las tecnologías que he seleccionado:

3.1. Frontend: Vue.js y Vite

Para la parte visual he utilizado Vue.js 3. Lo elegí porque permite construir la web mediante "componentes" reutilizables, lo que ayuda a que el código esté mucho más organizado. Además, al ser una Single Page Application (SPA), la navegación es fluida y no requiere recargas constantes. Para la construcción del proyecto uso Vite, que agiliza mucho el trabajo al mostrar los cambios en el navegador casi al instante.

3.2. Backend: Django y Python

El servidor corre sobre Python con el framework Django y Django REST Framework. No utilizo Django para renderizar plantillas HTML, sino como una API: recibe peticiones de Vue, procesa la lógica y devuelve los datos en formato JSON. Lo elegí principalmente por la seguridad que ofrece de serie y lo bien que gestiona el sistema de usuarios y permisos.

3.3. Base de Datos: PostgreSQL

Al manejar datos críticos como transacciones de dinero y stock, necesitaba una base de datos altamente fiable. PostgreSQL es una solución profesional que garantiza que la información de los pedidos y los encargos personalizados esté siempre íntegra y bien relacionada.

3.4. Pagos y emails

Para los cobros he integrado Stripe. Es la pasarela de pagos líder y me permite procesar tarjetas de forma segura sin que los datos bancarios toquen mi base de datos. Para los correos automáticos (como el envío de recibos), utilizo Mailtrap, que me ha servido para testear todo el flujo de emails durante el desarrollo sin enviar correos reales por error.

4. Herramientas empleadas

Estas son las herramientas que he usado en mi día a día para programar la plataforma:

4.1. Visual Studio Code

Es mi editor principal. Aquí es donde he redactado todo el código. Gracias a sus extensiones, puedo mantener un formato limpio y detectar fallos de sintaxis rápidamente.

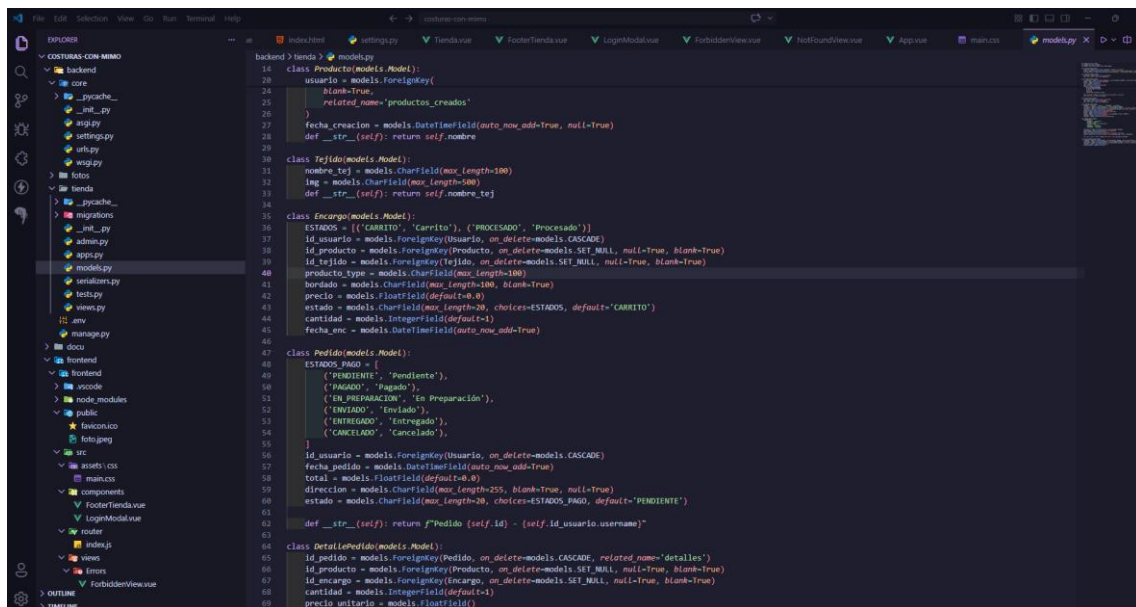


Ilustración 1 Visual Studio Code

4.2. pgAdmin 4

Es la herramienta visual que uso para administrar la base de datos PostgreSQL. Me ha permitido comprobar que las tablas se creaban correctamente y verificar que los datos de las clientas se guardaban bien tras cada compra.

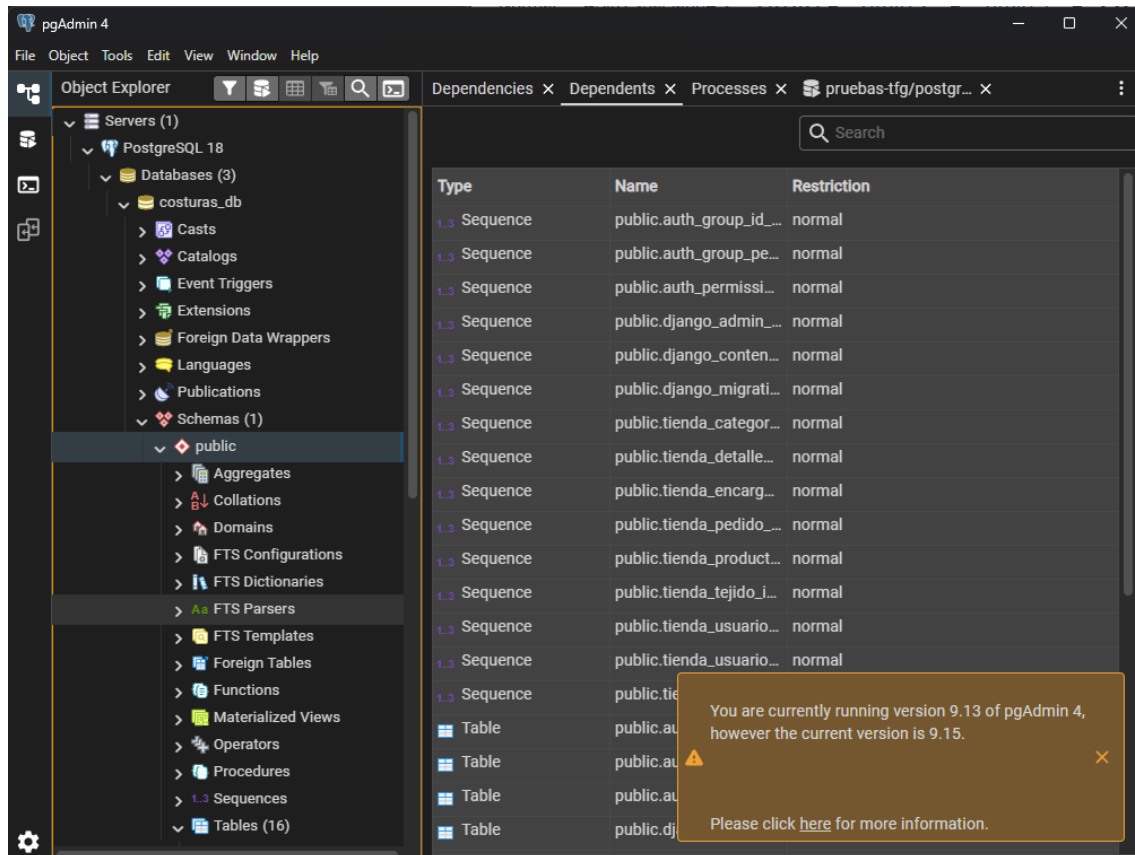


Ilustración 2 pgAdmin 4

5. Organización de carpetas

He organizado el proyecto en dos directorios principales para separar responsabilidades:

5.1. Frontend

En esta carpeta reside todo el código de la "cara visible" de la web. He organizado el proyecto siguiendo la estructura recomendada para Vue 3, separando la configuración global de los elementos visuales.

- main.js: Es el punto de arranque de la aplicación. Aquí es donde inicializo Vue, importo los estilos globales de la tienda y cargo las herramientas necesarias para que la web funcione, montando todo el sistema sobre el archivo HTML principal.

- `App.vue`: Es el componente raíz. He diseñado aquí la estructura general de la web para que sea totalmente responsive. Contiene el menú de navegación y el pie de página, de forma que se mantienen fijos mientras el contenido central cambia dinámicamente según la sección donde esté la clienta.
- `router/index.js`: Es el archivo que gestiona todas las URLs de la página. No solo sirve para movernos por la web, sino que aquí he programado la lógica de seguridad para proteger el panel de administración.
- Carpeta `views/`: Aquí guardo los componentes que representan páginas completas. He creado archivos separados para la `Tienda.vue`, el `Carrito.vue`, el `Perfil.vue` y el `AdminView.vue` (el panel de mi prima). Al estar separadas, el código es mucho más fácil de localizar y editar.
- Carpeta `components/`: En este directorio almaceno las piezas pequeñas y reutilizables de la interfaz. Por ejemplo, las tarjetas de los productos, los botones personalizados o el pie de página. Esto me permite hacer cambios en un solo archivo y que se apliquen en toda la web a la vez.

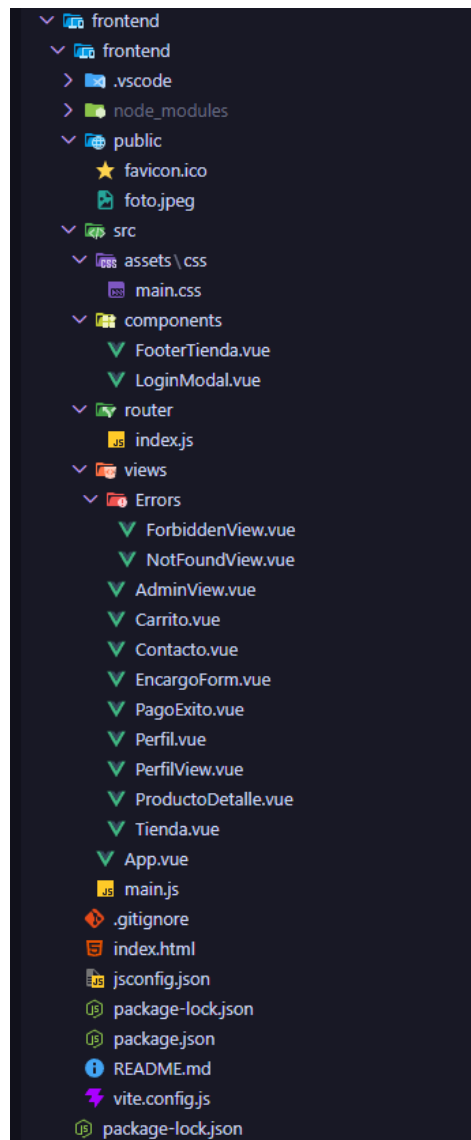


Ilustración 3 Organización Frontend

5.2. Backend

Estructura modular típica de Django:

- core/: Configuración central del servidor, incluyendo las llaves secretas y la conexión a la base de datos.
 - settings.py: Es el centro de configuración del servidor. Aquí he definido la conexión a la base de datos PostgreSQL, las llaves secretas de Stripe y los parámetros SMTP para que Mailtrap intercepte los correos de prueba.

- `urls.py`: Actúa como el enrutador principal. Se encarga de dirigir todas las peticiones que llegan desde la web hacia la lógica de la tienda y permite que las imágenes del taller se sirvan correctamente.
- `tienda/`: es el nombre que le he dado a la aplicación, por lo que es el nombre de donde se van a alojar todos los archivos relacionados con este proyecto.
 - `Admin.py`: En este archivo registro los modelos para tener acceso al panel interno de Django. Es mi "puerta trasera" para gestionar los datos rápidamente si hace falta, independientemente del panel de Vue.
 - `Models.py`: Es el corazón de los datos. Aquí he diseñado las tablas (Productos, Tejidos, Pedidos y Encargos) y cómo se relacionan entre ellas mediante claves foráneas para que no haya errores.
 - `views.py`: Aquí reside la lógica de las funciones. Controla qué ocurre cuando alguien intenta comprar, valida los pagos con Stripe y dispara los correos automáticos tras un pedido exitoso.

- serializers.py: Funcionan como traductores. Convierten la información compleja de la base de datos de Python a formato JSON, que es el único lenguaje que el frontend de Vue entiende.

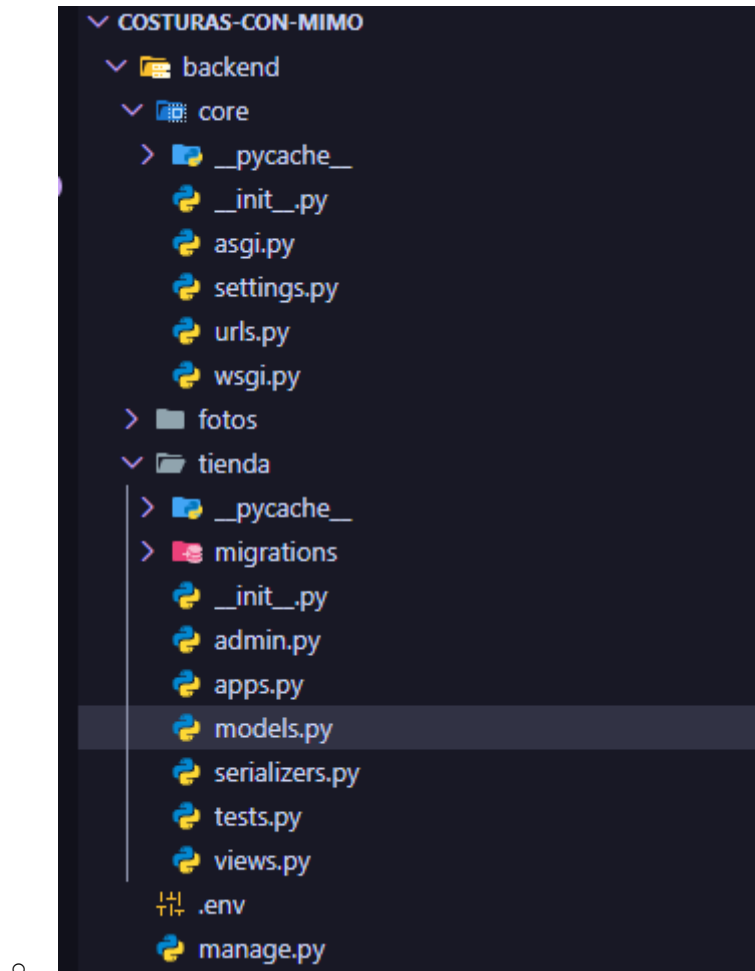


Ilustración 4 Organización Backend

6. Diseño de Datos

He diseñado el esquema de la base de datos buscando que la información esté siempre conectada de forma lógica:

- Usuario: Gestiona accesos y diferencia si el usuario es cliente o administrador.
- Producto: Almacena el nombre, foto, stock y precio del catálogo base.

- Tejido: Guarda el muestrario de telas que mi prima tiene disponibles en el taller.
- Pedido: Registro de la compra (total pagado, fecha y estado).
- Encargo: La tabla clave para la personalización, vinculando un producto con un tejido y el texto a bordar.

```
class Encargo(models.Model):
    ESTADOS = [('CARRITO', 'Carrito'), ('PROCESADO', 'Procesado')]
    id_usuario = models.ForeignKey(Usuario, on_delete=models.CASCADE)
    id_producto = models.ForeignKey(Producto, on_delete=models.SET_NULL, null=True, blank=True)
    id_tejido = models.ForeignKey(Tejido, on_delete=models.SET_NULL, null=True, blank=True)
    producto_type = models.CharField(max_length=100)
    bordado = models.CharField(max_length=100, blank=True)
    precio = models.FloatField(default=0.0)
    estado = models.CharField(max_length=20, choices=ESTADOS, default='CARRITO')
    cantidad = models.IntegerField(default=1)
    fecha_enc = models.DateTimeField(auto_now_add=True)

class Pedido(models.Model):
    ESTADOS_PAGO = [
        ('PENDIENTE', 'Pendiente'),
        ('PAGADO', 'Pagado'),
        ('EN_PREPARACION', 'En Preparación'),
        ('ENVIADO', 'Enviado'),
        ('ENTREGADO', 'Entregado'),
        ('CANCELADO', 'Cancelado'),
    ]
    id_usuario = models.ForeignKey(Usuario, on_delete=models.CASCADE)
    fecha_pedido = models.DateTimeField(auto_now_add=True)
    total = models.FloatField(default=0.0)
    direccion = models.CharField(max_length=255, blank=True, null=True)
    estado = models.CharField(max_length=20, choices=ESTADOS_PAGO, default='PENDIENTE')

    def __str__(self): return f"Pedido {self.id} - {self.id_usuario.username}"
```

Ilustración 5 Models.py

7. Explicación de código

Aquí detallo el funcionamiento de los procesos más importantes de la aplicación:

7.1. Login y seguridad

Para proteger el panel de administración, configuré un "guardián" en router/index.js. Antes de cargar cualquier página de gestión, el código comprueba si el usuario tiene un token de administrador. Si no es así, bloquea el acceso y te redirige a la tienda automáticamente.

```
router.beforeEach((to, from) => {  
  const token = localStorage.getItem('token');  
  const esAdmin = localStorage.getItem('isAdmin') === 'true';  
  if (to.meta.requiresAuth && !token) {  
    return '/';  
  }  
  
  if (to.meta.requiresAdmin && !esAdmin) {  
    return '/403';  
  }  
  
  return true;  
});  
  
export default router
```

Ilustración 6 Método index.js

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'tienda',
      component: TiendaView,
    },
    {
      path: '/producto/:id',
      name: 'producto-detalle',
      component: () => import('../views/ProductoDetalle.vue'),
      props: true
    },
    {
      path: '/encargo-personalizado',
      name: 'encargo',
      component: () => import('../views/EncargoForm.vue')
    },
    {
      path: '/carrito',
      name: 'carrito',
      component: () => import('../views/Carrito.vue')
    },
    {
      path: '/perfil',
      name: 'perfil',
      component: () => import('../views/PerfilView.vue'),
      meta: { requiereAuth: true }
    },
    {
      path: '/admin',
      name: 'admin',
      component: () => import('../views/AdminView.vue'),
      meta: { requiereAuth: true, requiereAdmin: true, ocultarNav: true }
    },
  ],
})
```

Ilustración 7 Rutas

7.2. Encargos

Programé el modelo de Encargo para que dependa obligatoriamente de un producto base y una tela específica. A nivel técnico, esto lo logré usando claves foráneas (ForeignKey) en Django. Esto bloquea cualquier intento de realizar un pedido personalizado si falta alguno de estos elementos. De esta forma, me aseguro de que mi prima recibe exactamente lo que necesita para ponerse a coser, sin pedidos fantasma ni datos a medias en la base de datos.

```
class EncargoViewSet(viewsets.ModelViewSet):
    queryset = Encargo.objects.all()
    serializer_class = EncargoSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Encargo.objects.filter(id_usuario=self.request.user)

    def perform_create(self, serializer):
        serializer.save(id_usuario=self.request.user)

    @action(detail=False, methods=['post'])
    def finalizar_pedido(self, request):
        user = request.user
        items_cesta = Encargo.objects.filter(id_usuario=user, estado='CARRITO')

        if not items_cesta.exists():
            return Response({'error': 'Cesta vacía'}, status=400)

        total_pago = sum(i.precio * i.cantidad for i in items_cesta)
        nuevo_pedido = Pedido.objects.create(id_usuario=user, total=total_pago)
```

Ilustración 8 Encargo View.py

7.3. Pago con Stripe

Cuando la clienta confirma su carrito de la compra, el frontend se comunica directamente con Stripe. Si el pago es aceptado por la entidad bancaria, el sistema recibe un token de confirmación que dispara la creación del pedido en mi base de datos y descuenta el stock de forma automática. De esta forma, evito almacenar tarjetas de crédito en mis propios servidores y garantizo una transacción altamente fiable.

```
const irAPagar = async () => {
    pagando.value = true
    try {
        const response = await axios.post('http://127.0.0.1:8000/api/encargos/finalizar_pedido/')
        if (response.data.url) {
            window.location.href = response.data.url
        }
    } catch (error) {
        alert("Error al conectar con la pasarela.")
    } finally {
        pagando.value = false
    }
}
```

Ilustración 9 Pago carrito.vue

7.4. Correos con mailtrap

Para mejorar la experiencia del usuario, tras un pago exitoso, Django ejecuta una función interna que conecta con el servidor SMTP de Mailtrap. Esta función envía un email en formato HTML: la cliente recibe su comprobante de compra y la administradora recibe un aviso de nuevo pedido, automatizando una tarea que antes se hacía manualmente por mensaje directo.

```
def perform_update(self, serializer):
    estado_anterior = serializer.instance.estado
    pedido = serializer.save()
    if estado_anterior != pedido.estado:
        self.enviar_correo_actualizacion(pedido)

def enviar_correo_actualizacion(self, pedido):
    asuntos = {
        'PAGADO': 'Tu pedido ha sido confirmado 🎉',
        'EN_PREPARACION': '¡Estamos preparando tu pedido! 🍳',
        'ENVIADO': '¡Tu pedido va en camino! 🚚',
        'ENTREGADO': 'Pedido entregado. ¡Disfrútalo! 📦',
        'CANCELADO': 'Tu pedido ha sido cancelado ❌'
    }
    asunto = asuntos.get(pedido.estado, f"Actualización de tu pedido #{pedido.id}")

    mensaje = f"""
    ¡Hola {pedido.id_usuario.username}!

    Te escribimos desde Costuras con Mimo para avisarte de que tu pedido #{pedido.id} ha cambiado de estado.

    Nuevo estado: {pedido.estado.replace('_', ' ')}
    Total del pedido: {pedido.total}€

    Puedes revisar los detalles de tu compra en tu perfil en cualquier momento:
    http://localhost:5173/perfil

    ¡Muchas gracias por confiar en nuestro taller artesanal!
    """

    try:
        send_mail(
            subject=asunto,
            message=mensaje,
            from_email='hello@demomailtrap.co',
            recipient_list=[pedido.id_usuario.email],
            fail_silently=False,
        )
    except Exception as e:
        print(f"Error al enviar el correo: {e}")
```

Ilustración 10 Envío de correos

8. Problemas

Durante el desarrollo me enfrenté a varios retos técnicos que me obligaron a investigar y realizar ajustes en la arquitectura inicial:

- Bloqueo por políticas CORS: En las primeras pruebas, el navegador bloqueaba las peticiones entre Vue y Django por estar ejecutándose en

puertos distintos (3000 y 8000). Lo solucioné instalando la librería `django-cors-headers` y configurando el backend para permitir el tráfico proveniente del frontend.

- **Visibilidad de las imágenes:** Conseguir que las fotos de las telas subidas desde el panel de control se vieran en la web pública requirió configuración extra. Tuve que definir las variables `MEDIA_URL` y `MEDIA_ROOT` en Django para que el servidor permitiera el acceso a esos archivos estáticos.
- **Reactividad del Panel de Control:** Quería que la gestión fuera muy ágil. Utilicé el sistema de estados reactivos de Vue para que, al cambiar el estado de un pedido (por ejemplo, a "Enviado"), la interfaz se actualizara al instante sin necesidad de recargar la página entera.

9. Puesta en marcha

Para desplegar y ejecutar este proyecto en un entorno local, los pasos a seguir desde la terminal son los siguientes:

1. Servidor Backend (Django):

- Ubicarse en la carpeta del backend.
- Arrancar el servidor API con: `python manage.py runserver`

2. Servidor Frontend (Vue.js):

- Ubicarse en la carpeta del frontend.
- Levantar el servidor de desarrollo ejecutando: `npm run dev`